AI-Assisted Development Evidence

Math Practice Web Application | Selected real prompts, debugging exchanges and test results | October 2026

This evidence document uses excerpts from the student–ChatGPT project conversation and results supplied by the student. Quotes marked as excerpts are shortened; descriptions of responses are summaries, not invented transcripts. The complete chat and Git history provide additional context.

1. Initial implementation request | Phase 3

Student prompt excerpt:

> "Implement Phase 3 of the Math Practice application using the provided specification and TODO list. Add operation selection, difficulty levels, and session-length options. Preserve existing functionality, write appropriate tests, and update the TODO list."

AI response and evaluation: ChatGPT supplied a React project update with configuration controls, tests and revised TODO/development log. The student ran local checks and reported a missing `@testing-library/user-event` dependency, which prevented tests and build. Installing the package resolved the issue; the student reported subsequent checks passed.

2. Detailed implementation request | Revised Phase 6

Student prompt excerpt:

> "Implement the revised Phase 6: Interactive Practice System ... Preserve all existing functionality ... Prevent duplicate result recording ... Do not implement persistent data storage, the complete statistics dashboard, or mistake-review algorithms."

AI response and evaluation: ChatGPT implemented question cards, session tracking and summaries. Student terminal output showed **49/50 tests passed**. The failing test rendered multiple *I Got It Wrong* buttons due to insufficient test cleanup. A cleanup hook resolved that test isolation issue, while a separate unused callback parameter was removed to fix linting.

3. Debugging with concrete error evidence | Phase 8

Student-supplied evidence: The test run showed **82/84 tests passed**, while ESLint and TypeScript flagged the expression below:

```
saveDifficulty, loadPractice(difficulty);
```

Correction: ChatGPT identified the malformed expression and recommended `saveDifficulty(difficulty);`. The other failures involved a multiple-alert query and an outdated expectation that mistake review was unsupported. The student also independently verified that recorded mistakes could be practiced again.

4. Requirement interpretation | Self-reported correctness

Student observation: After selecting 20 for 5 + 5 and clicking *I Got It Right*, the correct counter increased. The student asked ChatGPT to check the actual assignment requirements. The software specification states that the learner decides whether the attempt is recorded as right or wrong. The behavior was preserved to comply with the assigned self-reported scoring design.

5. Regression testing and refinement | Phase 9

Student-reported test output: 85/87 tests passed and both linting and the production build succeeded. One test still queried a selected answer by its old accessible name, although the UI now displayed *5 ✓ Selected*. Another direct-route test unexpectedly restored a saved session. Test assertions and storage setup were corrected without removing coverage. The student later reported no errors.

6. Final verification, audit and release

Student PowerShell results: After reinstalling dependencies in the final project, the student provided output showing **17 test files passed, 87/87 tests passed**, followed by successful ESLint and Vite production build. The subsequent Prettier check warned about 47 files. An npm audit identified three findings affecting the Vitest testing toolchain. These were disclosed rather than automatically applying a potentially breaking forced upgrade.

Manual and deployment checks: The student verified statistics reset, mistake review and mastery behavior, pushed the project to an existing class GitHub repository, deployed the project on Netlify, and reported that the deployed application worked. The assistant did not independently verify a public URL.

Optional supporting screenshots

For stronger direct evidence, append genuine screenshots of: a scoped Phase 6 prompt, the Phase 8 TypeScript error and correction, the final **87/87** terminal result, and the working Netlify application. Do not fabricate or recreate screenshots of conversations.